

Overview

Visualise is a personal cash-flow forecaster that helps users understand the flow of their money through income and expenses entries. By logging income and recurring expenses, the app simulates every day of the current month and provides the date in which the user's balance is lowest. Unlike other budget trackers, Visualise models overlapping billing cycles with recurrence types like daily, weekly, monthly, yearly, and custom intervals. This is to give an accurate picture of the financial situation of the user as to their net monthly total, monthly expenses and income.

Planning Approach

The project followed a deliberate top-down process before any code was written. Requirements were gathered first following SDLC. Defining the problem, the user, and the must-have features versus nice-to-haves. System design followed — database schema, selection of tech stack, and architecture decisions were decided before implementation. During implementation, the backend was built first and fully tested with Thunder Client before implementation of the frontend, ensuring the API contract was stable and accurate before the UI was integrated with it. Therefore, frontend integration came last, which was connecting real API responses to the chart and forms.

Tech Stack

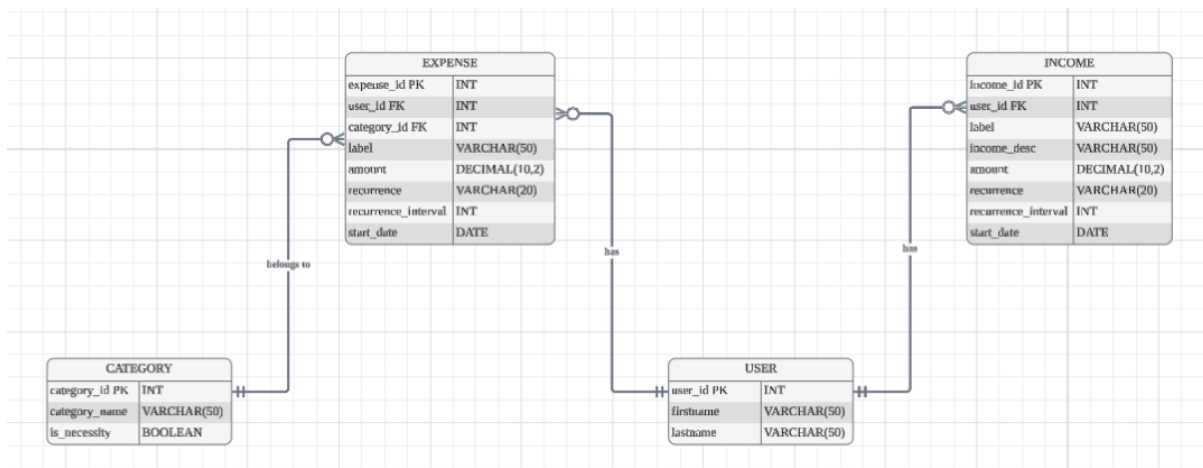
- React + TypeScript (Frontend) – Allows for app to be built with reusable components with TypeScript catching mistakes before it runs.
- FastAPI (Backend) – Fast and straightforward, it handles requests behind the scenes.
- PostgreSQL (Database) – The data is stored in an organised manner.
- SQLAlchemy – Allows developers to use Python which talks to PostgreSQL.
- Recharts – Turns data into charts and works well with React.

Architecture Overview

User enters data in the frontend. React validates inputs client-side by checking data is sensible and present in required fields. It then sends the data to FastAPI through an HTTP POST request. FastAPI validates the incoming data a second time against the schema and writes it to PostgreSQL with SQLAlchemy.

When the user opens the forecast view, React sends a GET request to FastAPI with the current month. This triggers the flux engine which fetches all income and expenses for the user from PostgreSQL and simulates every day of the month. For each day it checks whether any income or expense lands on that date based on its recurrence type, updates the user balance, and records the result. The engine returns a JSON object containing the daily balance history, financial events (days when expenses or income is expected), monthly income and expense, and the lowest balance day. React feeds the data it receives into Recharts to render the relevant charts (donut and line), with the lowest balance day marked and labelled. The pages are always recalculated fresh from stored data.

Database Schema



Key Technical Decisions

Centralised cash-flow calculation in the backend: All recurrence and forecast calculations are handled exclusively by the Python backend flux engine. Keeping calculation logic in one place which prevents duplicate logic and inconsistent results. An earlier version of the frontend from Figma AI contained its own TypeScript calculation engine which was defective and removed. The implementation uses the backend engine as the single source of truth.

Daily balance forecasting: The flux engine calculates the running balance for the user every day of the month. A daily simulation identifies the exact day the balance hits its lowest point, which is the core requirement of the app.

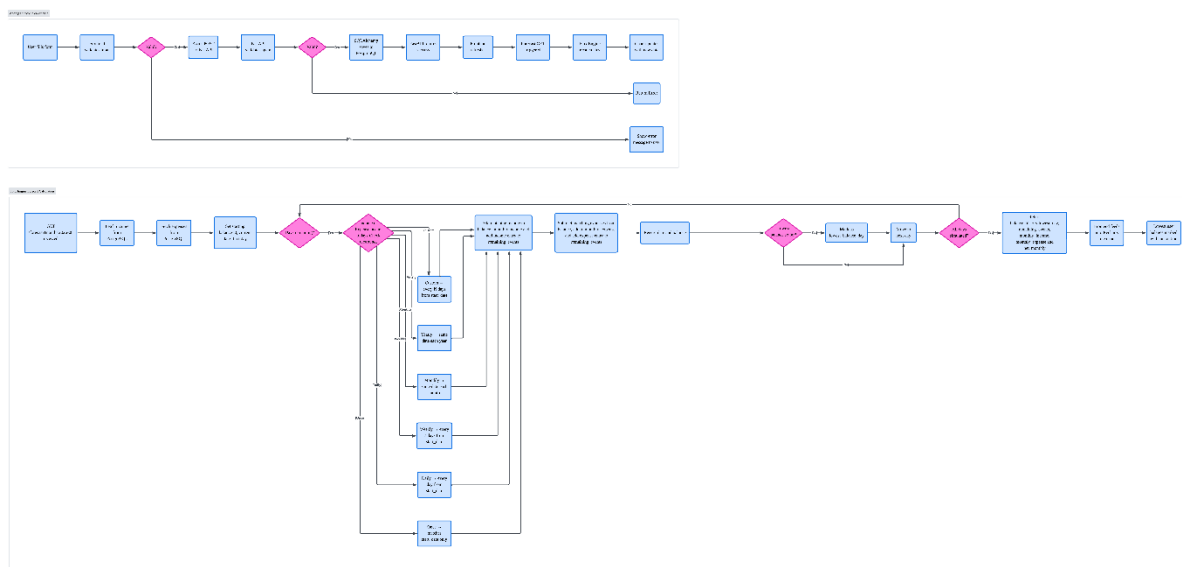
Income and Expense as separate tables: A single Transaction table was initially considered because both tables have similar attributes but ultimately rejected. Expenses have attributes category and necessity which income does not. Keeping them separate makes the data model clearer and implementation easier for the flux engine.

REST API separation between frontend and backend: React communicates with FastAPI exclusively through defined API endpoints with financial.service.ts that keeps the frontend API calls in one place. This allows for clear separation of frontend and backend logic, allowing both sides to be updated independently.

Recurrence Types: For Income and Expenses, users can pick the recurrence type like Daily, Weekly, Monthly, Yearly, Once and Custom. This to account for all types of recurrence payments and all cases are considered and implemented in the flux engine.

Flowcharts

PDF of flowchart can be found on my Github repo in docs folder.



AI was used throughout this project and verified before being shipped. Using Claude mainly for language syntax, boilerplate code, thought partner for ideas and considerations. Used ChatGPT also for language syntax and understanding behaviour of the application scenario like how monthly payments should work.

Claude was used as a thinking and planning tool throughout the design and backend phases. I used it as a sparring partner to consider and develop requirements, schema decisions and architecture. It also helped clarify syntax and implementation approaches when I was unsure of implementation for FastAPI and SQLAlchemy. In all cases the goal was understanding, not copying — each suggestion was analysed and developed until the logic was clear and correct.

Known Limitations

No historical forecast accuracy: When an income or expense is deleted, past months will recalculate as if it never existed as the forecast data is always recalculated when called upon. The solution that was considered is a snapshot table that freezes each month's forecast data when the month ends.

No necessity vs liability classification on expenses: The category model includes an is_necessity field and the data is seeded correctly, but the frontend form does not currently present this to the user, and the donut chart does not break down spending by necessity vs liability as planned in the Figma design and was also a feature I planned to include but ultimately scoped out due to time.

Monthly recurrence is fixed calendar date only: An expense set to monthly always recurs on the same calendar date. For example, if expense or income occurs on the 31st, if next month is only up to 30 days then the recurrence occurs on the 30 similarly for February the recurrence will occur on the 28.

Single currency: The app assumes Malaysian Ringgit. No currency selection or exchange rate is implemented. This was a nice to have functionality that I wanted to implement because users can use the application overseas and track spending but was scoped up due to time constraints.

Stress Tests: Due to time constraints comprehensive stress tests were unable to be properly done but extensive testing of backend logic was done with Thunder Client. Much of the tests were targeted to the flux_engine which dealt in all calculation and recurrence logic.